JHB DATA & AI SATURDAY · 31/10/2026

From Data to API in Minutes

Getting Started with Microsoft Fabric API for GraphQL



Sunelle Botha · Nico Botes

Session summary

Microsoft Fabric provides a fully native built-in “API for GraphQL” that automatically generates a GraphQL endpoint and schema from your Fabric data sources (Lakehouse, Warehouse, SQL DB), letting applications query Fabric data through a single GraphQL API without writing backend code.

This removes the need to build custom REST API’s or maintain SQL connections.

A single endpoint, supporting queries and mutations (CRUD).

All through an interactive editor, and optional type-safe clients.



Meet your speakers



Nico Botes

Today: GraphQL concepts and the Fabric architecture



Sunelle Botha

Today: live demo, showing you the concept in Fabric

THANKS TO THE SPONSORS

Days like today are only possible because of them.



CobaltAnalytics

Microsoft

MOYO
heart of digital change

AstuteTech Data

REAL PEOPLE • REAL INSIGHTS • A STRONGER COMMUNITY

50 minutes, mostly live



01

Why it matters

Modern data access and the trouble with hand-built APIs

02

GraphQL and Fabric

Fundamentals, GraphQL vs REST, architecture, use cases

03

Live demo

Create, query, relate, secure and integrate an API

04

Wrap-up and Q&A

Limits, lessons learned, resources and your questions

Everyone wants your Fabric data now

The governed, curated data lives in Fabric.
The list of things that need it keeps growing, and very few of them are reports.

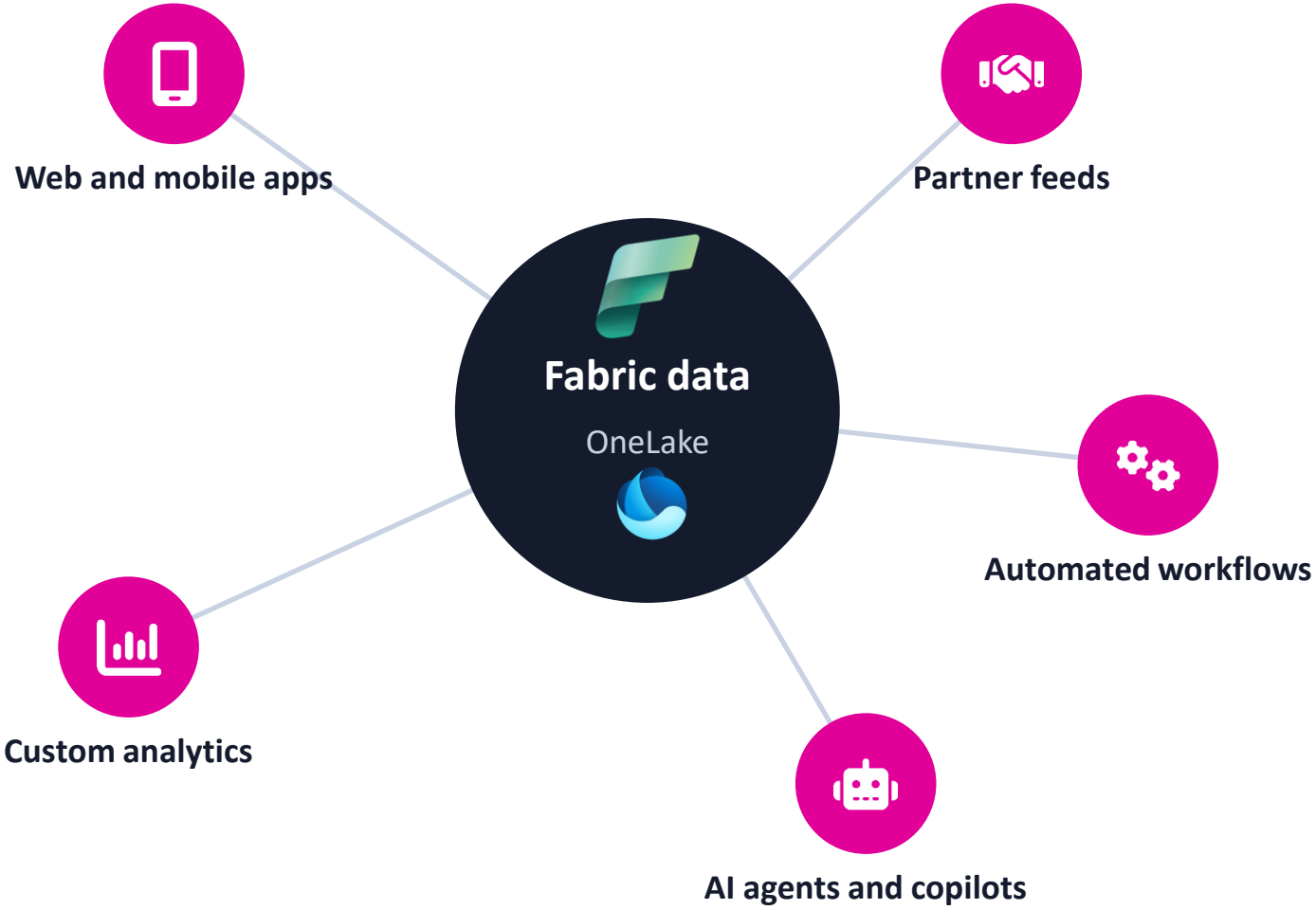
Every consumer wants the same thing:

- Fast
- Secure and governed
- Exactly the data they need, nothing more


Application & Integration Developers


BI Developers & Data Engineers


Data Scientists



The usual answers both cost you

Direct database connections

- Apps are tightly coupled to the schema, so a renamed column breaks them
- Every app manages its own drivers, connection strings and credentials
- SQL buried in app code is hard to test and maintain

Custom REST APIs

- Someone builds and runs a backend: controllers, routing, data access
- Data changes mean v1, v2, v3 of the API
- Fixed endpoints over-fetch or need several round trips

What if Fabric generated the API layer for you?
What if that minimizes the time applications take to access your data in Fabric?
That's today's session.

GraphQL solves these challenges



No backend code required

Fabric automatically generates the GraphQL schema, resolvers, and endpoint from your data sources



Query exactly what you need

Request specific fields in a single query, eliminating over-fetching and reducing payload sizes



Fetch related data in one request

Traverse relationships without multiple round trips (no N+1 query problem)



No-break Schema evolution

Add new fields without affecting existing queries—clients only request what they know about



Type safety and documentation

Self-documenting schema with built-in introspection that development tools understand



Unified access to multiple sources

Query across lakehouses, warehouses, and SQL databases through a single endpoint

02

What is Fabric API for GraphQL?

GraphQL fundamentals · GraphQL vs REST · Architecture · Use cases





The Facts

- Open-source query language & runtime maintained by GraphQL Foundation
- Created by Facebook in 2012 to overcome data-fetching inefficiencies in iOS & Android Mobile Apps
- Open-sourced in 2015 and moved to GraphQL Foundation, hosted by Linux Foundation in 2018
- GraphQL in Microsoft Fabric – Public Preview in May 2024, Generally Available in November 2024
- Both GraphQL & REST API are architecture styles – Graph becoming more-and-more popular
- GitHub GraphQL API is probably one of the best examples in use
- Data Governance Tools like Collibra heavily use GraphQL
- GraphQL is not a database – it needs to have a GraphQL Server hosting API accessible over HTTP
- Different than Graph Database but based on similar ideas and concepts

Read more about GraphQL and the Foundation here: https://graphql.org

GraphQL vs REST

	REST	GraphQL
Endpoints	One URL per resource	One endpoint for everything
Who shapes the data	The server decides what comes back	The client asks for exactly the fields it needs
Related data	Often several round trips	Nested in one request
Evolving the API	New versions: v1, v2, v3	Add fields, deprecate old ones, no versions
Contract	Optional (e.g. OpenAPI)	Strongly typed schema with introspection
Caching	HTTP caching works out of the box	Needs a gateway or client-side caching

REST isn't going anywhere. GraphQL shines when many clients need different slices of related data.

One screen, one request

A customer page: name, recent orders, and the products on each order

REST

1	`GET /customers/29485`	full payload
2	`GET /customers/29485/orders`	full payload
3	`GET /orders/71774/lines`	full payload

Three round trips, and every response carries fields the page never shows

GraphQL

```
query {
  customers_by_pk(CustomerID: 29485) {
    FirstName
    LastName
    orders {
      items {
        OrderDate
        lines { items { ProductID OrderQty } }
      }
    }
  }
}
```

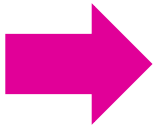
One round trip, only the fields on screen

= Reduce over-fetching & under-fetching of Data

Ask for a shape, get that shape back

YOUR QUERY

```
query {
  customers(first: 1) {
    items {
      FirstName
      LastName
      CompanyName
    }
  }
}
```



THE RESPONSE

```
{
  "data": {
    "customers": {
      "items": [{
        "FirstName": "Orlando",
        "LastName": "Gee",
        "CompanyName": "A Bike Store"
      }]
    }
  }
}
```

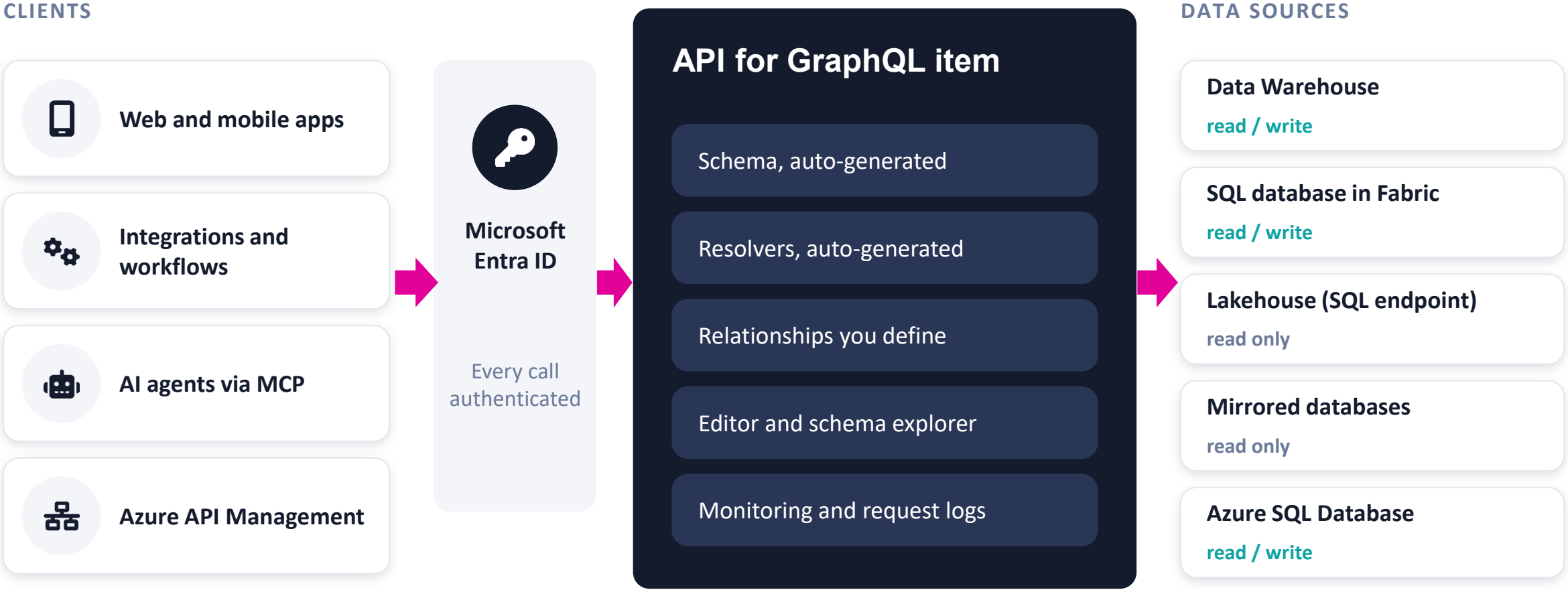
Three GraphQL operation types:

Query · read ✓

Mutation · write ✓

Subscription · not in Fabric

How Fabric API for GraphQL fits together



One endpoint, many sources: queries fan out in parallel. Relationships stay within a single source.

From data to API in five steps

1

Create the item

New item > API for GraphQL. Endpoint live at once

2

Connect sources

Pick from the OneLake catalog

3

Choose objects

Tables, views and stored procedures

4

Add relationships

Optional: one-to-one, one-to-many, many-to-many

5

Set permissions

Who can run queries and mutations

Fabric generates

- GraphQL schema
- Resolvers
- Endpoint URL
- Sample code

Your source decides read or read/write

Data source	Queries	Mutations
Fabric Data Warehouse	✓	✓
SQL database in Fabric	✓	✓
Azure SQL Database	✓	✓
Lakehouse, via SQL analytics endpoint	✓	—
Mirrored databases, via SQL analytics endpoint **SQL DB's – Azure, MI, Fabric, Snowflake, Open Mirror DB's, Azure Databricks etc.**	✓	—



Primary key required

Every exposed table or view needs one, or schema generation fails



Pick one connectivity mode

Single sign-on or saved credentials, for the whole API



Many sources, one endpoint

But no relationships or transactions across sources

Where it fits



Data-driven apps

Web and mobile apps read lakehouse and warehouse data without a custom backend

App developers



Share data safely

Expose curated data through a governed endpoint instead of handing out connection strings

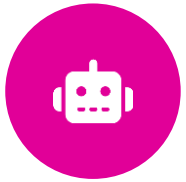
Data engineers



Custom analytics

Build analytics experiences that complement Power BI

BI developers



AI agents

A typed, introspectable schema lets agents discover the data and write their own queries

AI and integration developers

03

Live demo

- AdventureWorks in a SQL database in Fabric

1. Create the API
2. Generate and explore the schema
3. Query Fabric data
4. Relationships and nested queries
5. Security and governance
6. Apps and AI integration

Limits worth knowing on day one

100	100 s	64 MB	10	1,000
items per page by default	request timeout	maximum response	levels of query depth	objects per API item

Also: SQL analytics endpoint sources are read-only. Stored-procedure types skip paging, filtering, ordering and key lookups, and only the first result set is used.

Lessons learned

Warm it up

First call after idle is slower. Run a few queries before you measure or demo

Stay in one region

Client, capacity and data sources in the same Azure region

Expose only what's needed

Select objects, hide columns, shape data with keyed views

Logic lives in the database

Resolvers can't be customised: use views and stored procedures

Watch your capacity

Calls consume capacity. Cache repeat queries with APIM

Treat the API as code

Git, deployment pipelines and an exported SDL schema

Try it next week (or later today)

1. **Start a Fabric trial**
   Or use any workspace on a Fabric capacity

2. **Create a SQL database**
   Load the AdventureWorks sample

3. **Add an API for GraphQL**
   Querying in about ten minutes

4. **Connect an app or agent**
   Clone microsoft/fabric-samples

RESOURCES

Overview
learn.microsoft.com/fabric/data-engineering/api-graphql-overview

Get started
learn.microsoft.com/fabric/data-engineering/get-started-api-graphql

Connect applications
learn.microsoft.com/fabric/data-engineering/connect-apps-api-graphql

AI agents with MCP
learn.microsoft.com/fabric/data-engineering/api-graphql-local-model-context-protocol

Samples
github.com/microsoft/fabric-samples

SESSION FEEDBACK

Please use this QR code to complete the speaker feedback form

Each completed form gives you an entry to win one of our **awesome prizes!**

REAL PEOPLE • REAL INSIGHTS • A STRONGER COMMUNITY

04

Questions?



Thank you,
JHB Data & AI Saturday #4

SESSION FEEDBACK

Please use this QR code to complete the speaker feedback form

Each completed form gives you an entry to win one of our **awesome prizes!**

REAL PEOPLE • REAL INSIGHTS • A STRONGER COMMUNITY